

Bank Account Management System

Project Report

Submitted By: [Your Name / Roll No]

Course: [Course Name]

Date: [Submission Date]

2. Introduction

The Bank Account Management System is a lightweight, interactive Command Line Interface (CLI) application built using Core Java. It simulates essential banking operations, allowing users to create accounts, perform transactions (deposits and withdrawals), and check balances. This project serves as a practical implementation of Object-Oriented Programming (OOP) concepts, demonstrating data encapsulation, state management, and the use of Java Collections for in-memory data storage.

3. Problem Statement

Managing financial transactions manually is prone to calculation errors and inefficiencies. Furthermore, beginners learning software development need practical, relatable scenarios to understand how real-world data—like bank balances and transactions—are managed computationally. There is a need for a straightforward, lightweight simulation that models core banking operations without the overwhelming complexity of a full-scale enterprise application.

4. Functional Requirements

- **Account Creation:** The system must allow new users to open an account with an initial deposit and automatically generate a unique account number.
- **Deposit Functionality:** Users must be able to add funds to their specific account using their account number.
- **Withdrawal Functionality:** Users must be able to deduct funds. The system must validate that the withdrawal amount does not exceed the available balance.
- **Balance Inquiry:** The system must display current account details (Name, Account Number, and Balance) upon request.

5. Non-functional Requirements

- **Usability:** The CLI must be intuitive with a clear menu-driven interface.
- **Performance:** Operations must be executed instantly in memory without noticeable delay.
- **Reliability:** The application must handle basic logical errors, such as preventing negative deposits or overdrafts.
- **Portability:** Being Java-based, the system should run on any machine with a JVM installed.

6. System Architecture

The system follows a simple monolithic architecture suitable for a console application. It consists of two primary components interacting within the JVM:

- **Data Model layer (Account):** Handles the state and business logic of a single bank entity.
- **Controller/View layer (BankApp):** Manages user input via `java.util.Scanner`, controls the execution loop, and maintains a `HashMap` serving as the in-memory data repository linking unique Integer IDs to Account objects.

7. Design Diagrams

7.1 Class / Component Diagram

The following structure represents the OOP design of the system:

Account Class (Model)	BankApp Class (Controller)
Attributes: - int accountNumber - String name - double balance Methods: + Account(id, name, bal) + deposit(double amount) + withdraw(double amount) + displayAccount()	Attributes: - static Scanner sc - static HashMap<Integer, Account> accounts - static int nextAccountNumber Methods: + main(String[] args) (Contains the menu loop and switch-case logic)

7.2 ER Diagram (Storage Note)

Note on Storage: Since this application uses in-memory Collections (`HashMap`) rather than a Relational Database, a traditional ER diagram is simplified to a Key-Value pair structure:

- **Key:** Integer (Account Number)
- **Value:** Account Object (Contains Name, ID, Balance)

7.3 Use Case & Workflow Diagrams

System Workflow Flowchart (Text Representation)

[Start] → [Display Main Menu] → [Take User Input]

└ If 1: [Gather Name & Deposit] → [Instantiate Account] → [Save to Map]

└ If 2: [Input Acc No] → [Validate] → [Increase Balance]

└ If 3: [Input Acc No] → [Validate Balance] → [Decrease Balance]

└ If 4: [Input Acc No] → [Fetch from Map] → [Print Details]

└ If 5: [Exit Program]

[Loop Back to Main Menu until Exit]

8. Design Decisions & Rationale

- **Use of HashMap:** A `HashMap<Integer, Account>` was chosen for data storage because it allows constant-time $O(1)$ lookups. When a user wishes to transact, looking up their account by the account number is instant.
- **No Database / In-Memory Only:** To keep the scope appropriate for a foundational project, database connectivity (JDBC) was omitted. This reduces setup complexity and focuses the learning on core Java logic.
- **Separation of Concerns:** The banking logic (deposit validation, withdrawal math) is kept inside the `Account` class, not the `main` method. This adheres to OOP encapsulation principles.

9. Implementation Details

The code is structured in a single Java file to facilitate easy compilation. The core mechanism is a `do-while` loop that continuously displays a menu. A `switch-case` block routes the user's integer choice to the appropriate banking function.

A critical implementation detail was handling the `Scanner` buffer. After reading an integer (`sc.nextInt()`), a trailing newline character remains in the buffer. An explicit `sc.nextLine()` is executed during account creation to consume this newline, preventing input skipping errors when reading the customer's name.

10. Screenshots / Results

The following are simulated console outputs representing successful execution of the program's features.

10.1 Application Start and Account Creation

```
===== Welcome to Bank App =====  
  
1. Open Account  
2. Deposit Money  
3. Withdraw Money  
4. Check Account Details  
5. Exit  
Enter your choice: 1  
Enter your name: Jane Doe  
Enter initial deposit: 1500  
Account created successfully!  
Your account number is: 1001
```

10.2 Performing a Deposit

```
Enter your choice: 2  
Enter account number: 1001  
Enter amount to deposit: 500  
Deposit successful!  
New balance: 2000.0
```

10.3 Attempting an Overdraft (Validation Check)

```
Enter your choice: 3  
Enter account number: 1001  
Enter amount to withdraw: 5000  
Insufficient balance.
```

11. Testing Approach

The system was tested manually using a Black-Box approach, focusing on edge cases and invalid inputs within the bounds of expected data types.

- **Positive Testing:** Verified that correct inputs result in successful account creation and accurate mathematical updates to balances.
- **Negative Testing (Logic Constraints):** Tested withdrawing more than the balance to ensure the "Insufficient balance" block triggers. Tested negative initial deposits to ensure the system rejects them.
- **Account Lookup Testing:** Entered non-existent account numbers (e.g., 999) to verify the system safely reports "Account not found" rather than crashing with a `NullPointerException`.

12. Challenges Faced

- **Scanner Buffer Issues:** Early versions of the code skipped the name input prompt entirely. This was diagnosed as the `nextInt()` method leaving a newline in the input stream. Resolved by adding `sc.nextLine()` immediately after reading the menu choice.
- **Data Loss on Exit:** Because data is stored in a `HashMap`, exiting the program wipes all accounts. Acknowledging this limitation was a conceptual challenge, leading to the identification of File I/O for future scope.

13. Learnings & Key Takeaways

- **OOP in Practice:** Gained a concrete understanding of how objects (Accounts) hold state independently of one another.
- **Collections Framework:** Learned the practical syntax and advantages of using `HashMap` for storing and retrieving key-value pairs efficiently.
- **Control Flow:** Improved proficiency with infinite loops, exit conditions, and switch-case routing in CLI applications.

14. Future Enhancements

To upgrade this system from a simple simulation to a more robust application, the following features could be implemented:

- **Persistent Storage:** Integrating Java File I/O or a lightweight database (like SQLite or MySQL via JDBC) so that accounts and balances are saved permanently across sessions.
- **Input Exception Handling:** Wrapping `Scanner` inputs in `try-catch` blocks to handle `InputMismatchException`. Currently, typing a letter when a number is expected will crash the application.
- **Transaction History:** Creating an `ArrayList` within the `Account` class to keep a log of the last 10 transactions (amounts and types) for the user to view.
- **Security features:** Adding a PIN or password requirement during account creation and verifying it before allowing withdrawals.

15. References

- **Oracle Java Documentation:** Used for referencing `java.util.Scanner` and `java.util.HashMap` syntax and methods. (<https://docs.oracle.com/en/java/>)
- **Project Prompt / Problem Statement:** Original assignment requirements and architectural guidelines provided for the CLI system.
- **Stack Overflow:** Researched solutions for Java Scanner newline buffer skipping behavior.